

## **\*\* LAB\*\* Esercizi di scripting risolti - da svolgere in autonomia**

Sito: [Virtuale](#)

Corso: Amministrazione di Sistemi T - Laboratorio di  
Amministrazione di Sistemi T

Libro: **\*\* LAB\*\*** Esercizi di scripting risolti - da svolgere in autonomia

Stampato da: Lorenzo Bernardini

Data: martedì, 12 maggio 2026, 15:24

## Descrizione

Leggere e comprendere il codice.

Copiarlo ed eseguirlo per verificarne il funzionamento (salvare ogni script in un file, ad esempio **listafile.sh**, dare i permessi di esecuzione con **chmod +x listafile.sh**, eseguire con **./listafile.sh**)

Ricordare che aggiungendo l'opzione x a bash (quindi modificando la prima riga in **#!/bin/bash -x**) si attiva la modalità di tracciamento delle singole righe eseguite.

## Sommario

1. **ls** ricorsivo (test, cd, funzioni, cicli)
2. **ls** ricorsivo, alternative (while, read, filtri)
3. **ls** ricorsivo, estensioni (elif, \$@)
4. conversione e manipolazione del tempo
5. conversione e manipolazione del tempo - pianificazione eventi
6. conversione e manipolazione del tempo - esecuzione con cron
7. conversione e manipolazione del tempo - esecuzione con at

# 1. Is ricorsivo (test, cd, funzioni, cicli)

Scopo: implementare manualmente l'esplorazione ricorsiva di una cartella (equivalente all'opzione **-R** di **ls**)

Andiamo per passi:

**ciclo principale - iterazione sulla cartella corrente**

```
for filename in *          # bash cura l'espansione corretta dei nomi
do
    if test -d "$filename" ; then
        # esplora la directory, come?
    else
        ls -l "$filename"
    fi
done
```

L'esplorazione di una subdirectory è analoga al ciclo principale, quindi il codice può essere posto in una funzione (chiamiamola *esploraDirectoryCorrente*) e riutilizzato.

La funzione va invocata solo se la directory è accessibile, e la navigazione in ingresso e in uscita dalla subdirectory deve essere sempre corretta.

Possibilità:

step-by-step senza processi aggiuntivi

```
test -x "$filename" && {
    cd "$filename"
    esploraDirectoryCorrente
    cd ..
}
```

oppure con una subshell che racchiude test e cambio di directory, che oltre a condizionare l'esecuzione è valido solo localmente quindi non influente sul processo chiamante:

```
( cd "$filename" && esploraDirectoryCorrente )
```

risultato (usando il secondo approccio):

```
#!/bin/bash
function esploraDirectoryCorrente() {
    echo DIR "$1"
    for filename in *
    do
        if test -d "$filename" ; then
            ( cd "$filename" && esploraDirectoryCorrente "$filename" )
        else
            ls -l "$filename"
        fi
    done
}
```

```
# main
esploraDirectoryCorrente .
```

in alternativa si può invocare incondizionatamente la funzione e mettere la logica al suo interno

```
#!/bin/bash
function esploraDirectoryCorrente() {
    echo DIR "$1"
    cd "$1" || return 1

    for filename in *
    do
        if test -d "$filename" ; then
            esploraDirectoryCorrente "$filename"
        else
            ls -l "$filename"
        fi
    done
    cd ..
}

#main
esploraDirectoryCorrente .
```

## 2. Is ricorsivo, alternative (while, read, filtri)

Alternative per il ciclo **for filename in \***

Possono essere utili per usare un comando arbitrario che fornisca un elenco di file personalizzato. qui useremo come esempio **ls -t**, che dà l'elenco in ordine di tempo di ultima modifica

1) **SBAGLIATO!** cosa succede se ci sono nomi di file con caratteri speciali?

```
for filename in $(ls -t)
```

...

2) **CORRETTO:** pipeline che riconosce le righe (il parametro 1 forza l'output singola colonna)

```
ls -lt | while read filename
```

...

**Vediamo ora un'alternativa all'uso del comando test.**

NOTE: in questo caso specifico, questa soluzione è **SCONSIGLIATA**.

Viene presentata come esempio di processo del testo che permette di estrarre una molteplicità di informazioni con un singolo stream e minimizzando il numero di comandi che accedono al filesystem, ma ogni volta che si può usare un comando come **test** che riconosce direttamente una proprietà del sistema (in questo esempio, il tipo di file), è molto più sicuro utilizzare quello invece che dedurre la stessa proprietà sintatticamente da una stringa testuale.

Notate infatti come il trattamento sintattico e non semantico dei link simbolici crei due problemi:

- 1) non c'è garanzia che il separatore non compaia anche come parte dei nomi, per cui il parsing con **awk** potrebbe sbagliare.
- 2) bisogna distinguere a mano tra link a file (da elencare) e link a directory (da seguire), che **test -d** invece riconosce correttamente. Seguire i link inoltre potrebbe essere complesso in termini di navigazione.

**SUGGERIMENTO:** quando si deve processare uno stream di testo, riportare un esempio del formato come commento.

```
# output di ls -l
# -rwxr-xr-x 1 root root    147176 Sep 24  2020 vdir
# lrwxrwxrwx 1 root root         20 Dec 30 17:13 vi -> /etc/alternatives/vi
```

le righe quindi possono essere "parsate" con

```
read permessi nlink utente gruppo dimensione mese giorno oraanno nome
```

Per i symlink è necessario usare un "cut avanzato" separando nome e destinazione del link sulla stringa " -> "

Ulteriore problema: provate **ls -l** da solo, e notate la prima riga. **ls** non ha un parametro per sopprimere l'header, quindi dobbiamo prendere a mano dalla seconda in poi.

```
#!/bin/bash -x

function esploraDirectoryCorrente() {
    echo DIR "$1"
    cd "$1" || return 1

    ls -l | tail -n +2 | while read permessi nlink utente gruppo dimensione mese giorno oraanno nome
    do
        if [[ "$permessi" =~ ^d ]] ; then

            esploraDirectoryCorrente "$nome"

        elif [[ "$permessi" =~ ^l ]] ; then

            filename=$(echo "$nome" | awk -F ' ' -> ' '{ print $1 }')
            target=$(echo "$nome" | awk -F ' ' -> ' '{ print $2 }')
            if [[ $(ls -ld "$target") =~ ^d ]] ; then
                esploraDirectoryCorrente "$target"
            else
                ls -l "$filename"
            fi
        else

            ls -l "$nome"

        fi
    done
    cd -
}

# main
esploraDirectoryCorrente .
```

### 3. Is ricorsivo, estensioni (elif, \$@)

Lo script realizzato finora non accetta parametri per lavorare su più argomenti simultaneamente. Ecco una versione che implementa questa funzionalità.

```
#!/bin/bash

function esploraDirectory() {
    echo DIR "$1"
    cd "$1" || return 1
    for filename in *
    do
        if test -d "$filename" ; then
            esploraDirectory "$filename"
        else
            ls -l "$filename"
        fi
    done
    cd ..
}

# main
for D in "$@" ; do
    esploraDirectory "$D"
done
```



## 4. conversione e manipolazione del tempo

Il comando **date** può essere usato per estrarre il tempo corrente e per convertire il tempo tra diversi formati.

Questo permette anche di semplificare i calcoli: utilizzando la misura in secondi dall'epoca di Unix, si evita di dover tenere conto dei riporti tra ore, giorni, mesi di diversa durata, ecc.

Esempio: realizzare uno script che legga da un file **~/calendario** una lista di scadenze espresse nel formato

| YYYY-MM-DD HH:MM DESCRIZIONE |
|------------------------------|
|------------------------------|

e scriva sul file **~/promemoria** due avvisi, 1 giorno e 1 ora prima della scadenza.

Vediamo una prima versione che utilizza un file temporaneo per pianificare i task e verificarli periodicamente, cercandoli testualmente nel file:

```
#!/bin/bash
FILEALLARMI=$(mktemp)
cat calendario | while read DATA ORA DESCRIZIONE
do
    SCADENZA=$(date +%s -d "$DATA $ORA") # conversione in secondi dal 1970-01-01 01:00
    GIORNOPRIMA=$(( SCADENZA - 24*60*60 ))
    ORAPRIMA=$(( SCADENZA - 60*60 ))
    echo "$GIORNOPRIMA $DATA $ORA $DESCRIZIONE" >> "$FILEALLARMI"
    echo "$ORAPRIMA $DATA $ORA $DESCRIZIONE" >> "$FILEALLARMI"
done

# ho necessità di far eseguire il controllo una e una sola volta ogni minuto,
# una possibilità è ciclare più di frequente e controllare se il minuto cambia
# in questo modo compenso anche l'imprevedibilità dei tempi di esecuzione
# del corpo del ciclo (mentre se usassi sleep 60 potrei andare alla deriva)

NOW=0
while sleep 20 ; do
    [[ $(date +%M) -eq $NOW ]] && continue

    NOW=$(date +%M)

    # nota: i timestamp originali hanno la granularità del minuto,
    # ma il tempo corrente considera i secondi, devo troncarli
    # trick, ma si può fare in altri modi: la divisione per 60 restituisce un intero...
    CURRENTMINUTE=$(( $(date +%s) / 60 * 60 ))

    # nota: ci possono essere zero, una o più righe col timestamp corrente
    # grep | while è il modo più semplice di gestirle, un'iterazione per ogni riga
    # incluso il caso di zero iterazioni nei minuti per cui non ci sono corrispondenze
    egrep "^$CURRENTMINUTE" "$FILEALLARMI" | while read TIMESTAMP DATA ORA DESCRIZIONE ; do
        echo "$DESCRIZIONE scade il $DATA alle $ORA"
    done >> ~/promemoria
done
```

## 5. conversione e manipolazione del tempo - pianificazione eventi

Notate che la seconda parte dello script può essere in realtà eseguita separatamente e ogni minuto verifica l'intero file, che quindi può essere aggiornato concorrentemente

Estensione della prima parte (chiamiamola **plan.sh**) in modo che accetti due parametri: il calendario e il file delle scadenze. Esamina il calendario e accoda al file delle scadenze solo quelle che non sono già presenti. Introduciamo anche il concetto di controllo dei parametri.

```
#!/bin/bash

test -f "$1" || {
    echo il primo parametro deve essere un file calendario esistente
    exit 1
}

test "$2" || {
    echo servono due parametri, il secondo deve essere un nome di file delle scadenze da creare o aggiornare
    exit 2
}

CALENDARIO="$1"
FILEALLARMI="$2"

cat "$CALENDARIO" | while read DATA ORA DESCRIZIONE
do
    SCADENZA=$(date +%s -d "$DATA $ORA") # conversione in secondi dal 1970-01-01 01:00
    RIGAGIORNOPRIMA="$(( SCADENZA - 24*60*60 )) $DATA $ORA $DESCRIZIONE"
    RIGAORAPRIMA="$(( SCADENZA - 60*60 )) $DATA $ORA $DESCRIZIONE"
    grep -qx "$RIGAGIORNOPRIMA" "$FILEALLARMI" || echo "$RIGAGIORNOPRIMA" >> "$FILEALLARMI"
    grep -qx "$RIGAORAPRIMA" "$FILEALLARMI" || echo "$RIGAORAPRIMA" >> "$FILEALLARMI"
done
```

L'esecuzione dei task scritti nel file delle scadenze può essere fatta con uno script **remind.sh** costituito dalla seconda parte del codice originale, che prenda come parametro il nome del file

```
#!/bin/bash
test -f "$1" || {
    echo specificare il file delle scadenze come parametro
    exit 1
}
FILEALLARMI="$1"
NOW=0
while sleep 20 ; do
    [[ $(date +%M) -eq $NOW ]] && continue
    NOW=$(date +%M)
    CURRENTMINUTE=$(( $(date +%s) / 60 * 60 ))
    egrep "^$CURRENTMINUTE" "$FILEALLARMI" | while read TIMESTAMP DATA ORA DESCRIZIONE ; do
        echo "$DESCRIZIONE" scade il $DATA alle $ORA
    done >> ~/promemoria
done
```

## 6. conversione e manipolazione del tempo - esecuzione con cron

Utilizzando il demone cron si può evitare il ciclo.

Il primo script può accertarsi che il secondo venga eseguito da cron. Per fare ciò, estrae il file di configurazione attivo di cron, e prima di aggiungere la riga che schedula remind.sh, ripulisce il file di cron da eventuali righe che già siano presenti con la stessa direttiva, per evitare di duplicarle. Infine installa la nuova configurazione.

```
#!/bin/bash

test -f "$1" || {
    echo il primo parametro deve essere un file calendario esistente
    exit 1
}

test "$2" || {
    echo servono due parametri, il secondo deve essere un nome di file delle scadenze da creare o aggiornare
    exit 2
}

CALENDARIO="$1"
FILEALLARMI="$2"

cat "$CALENDARIO" | while read DATA ORA DESCRIZIONE
do
    SCADENZA=$(date +%s -d "$DATA $ORA") # conversione in secondi dal 1970-01-01 01:00
    RIGAGIORNOPRIMA="$(( SCADENZA - 24*60*60 )) $DATA $ORA $DESCRIZIONE"
    RIGAORAPRIMA="$(( SCADENZA - 60*60 )) $DATA $ORA $DESCRIZIONE"
    grep -qx "$RIGAGIORNOPRIMA" "$FILEALLARMI" || echo "$RIGAGIORNOPRIMA" >> "$FILEALLARMI"
    grep -qx "$RIGAORAPRIMA" "$FILEALLARMI" || echo "$RIGAORAPRIMA" >> "$FILEALLARMI"
done

TMPCRON=$(mktemp)
RIGACRON="* * * * * $HOME/remind.sh $FILEALLARMI"
crontab -l | grep -Fvx "$RIGACRON" > "$TMPCRON"
echo "$RIGACRON" >> "$TMPCRON"
crontab "$TMPCRON"
rm -f "$TMPCRON"
```

cron garantisce che **remind.sh** sia eseguito esattamente allo scadere di ogni minuto, quindi il ciclo e tutte le precauzioni annesse non servono più

```
#!/bin/bash
# NOTA: non dovrebbe mai succedere che il parametro sia errato
# visto che remind.sh è schedulato dall'altro script via cron
test -f "$1" || {
    echo specificare il file delle scadenze come parametro
    exit 1
}

FILEALLARMI="$1"
CURRENTMINUTE=$(( $(date +%s) / 60 * 60 ))
egrep "^$CURRENTMINUTE" "$FILEALLARMI" | while read TIMESTAMP DATA ORA DESCRIZIONE ; do
    echo "$DESCRIZIONE scade il $DATA alle $ORA"
done >> ~/promemoria
```

## 7. conversione e manipolazione del tempo - esecuzione con at

Questo scenario è in realtà più adatto all'uso di **at** che a quello di **cron**

Manteniamo la prima parte per aggiornare il calendario, senza bisogno di un file delle scadenze, mantenendo una copia di backup solo al fine di capire cosa era già presente e cosa è stato aggiunto.

Dopo aver calcolato "meno un giorno" e "meno un'ora" bisogna riconvertire i timestamp in formato "YYMMDDhhmm", quello più pratico in questo caso, tra i vari accettati da **at**.

L'intera logica è attuabile da un singolo script, ma i job schedulati con at allo stesso minuto rischiano di scrivere concorrentemente sullo stesso file, per cui è necessario utilizzare un lock. Il primo processo a lanciare **flock \$HOME/promemoria** acquisisce un lock esclusivo, se altri fanno lo stesso restano in attesa finché il lock non viene rilasciato.

```
#!/bin/bash

test -f "$1" || {
    echo il primo parametro deve essere un file calendario esistente
    exit 1
}

CALENDARIO="$1"
test -f "OLD-$CALENDARIO" || touch "OLD-$CALENDARIO"

cat "$CALENDARIO" | while read DATA ORA DESCRIZIONE
do
    SCADENZA=$(date +%s -d "$DATA $ORA")
    GIORNOPRIMA=$(date '+%Y%m%d%H%M' -d @$( ( SCADENZA - 24*60*60 ) ) )
    ORAPRIMA=$(date '+%Y%m%d%H%M' -d @$( ( SCADENZA - 60*60 ) ) )

    grep -qx "$DATA $ORA $DESCRIZIONE" "OLD-$CALENDARIO" || {
        at -t "$GIORNOPRIMA" <<< "flock $HOME/promemoria echo '$DESCRIZIONE scade il $DATA alle $ORA' >> $HOME/promemoria"
        at -t "$ORAPRIMA" <<< "flock $HOME/promemoria echo '$DESCRIZIONE scade il $DATA alle $ORA' >> $HOME/promemoria"
    }
done
cat "$CALENDARIO" > "OLD-$CALENDARIO"
```